

PROGRAMMAZIONE II

JAVA IO

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

STREAM DI IO



FLUSSO DI DATI IN INPUT E OUTPUT

In Java, le operazioni di IO si basano sull'astrazione dello **stream** (flusso di dati)

- ▶ Gli stream di IO sono diversi dall'interfaccia `Stream`

Uno stream di IO può essere collegato a

- ▶ Un file sul disco
- ▶ Standard input, output, error
- ▶ Una connessione di rete
- ▶ Un flusso di dati da/verso dispositivi hardware

// ad es. `System.in`

Le operazioni di IO funzionano allo stesso modo con tutti i tipi di stream di IO

FLUSSO DI DATI IN INPUT E OUTPUT (CONT.)

Fornito dal package `java.io`

- ▶ Le eccezioni correlate sono sottoclassi di `IOException`

Stream orientati ai byte

- ▶ Classi astratte `InputStream` e `OutputStream`
- ▶ Flusso di byte

Stream orientati ai caratteri

- ▶ Classi astratte `Reader` e `Writer`
- ▶ Flusso di caratteri Unicode a 16 bit

STREAM DI BYTE

`InputStream` e `OutputStream` definiscono canali di input e output basati su byte

- ▶ Il costruttore apre lo stream
- ▶ Il metodo `read()` ottiene uno o più byte dallo stream // `InputStream`
- ▶ Il metodo `write()` inserisce uno o più byte nello stream // `OutputStream`
- ▶ Il metodo `close()` chiude lo stream

Le implementazioni concrete sono specifiche del canale

- ▶ Tastiera standard, schermo, file, . . .

STREAM DI BYTE (CONT.)

Adatti per gestire dati binari grezzi come immagini, audio, bytecode, . . .

- ▶ `System.in` è un `BufferedInputStream`
- ▶ `System.out` è un `BufferedOutputStream`

Implementazioni concrete

`BufferedInputStream`

`DataInputStream`

`FileInputStream`

`BufferedOutputStream`

`DataOutputStream`

`FileOutputStream`

Legge dati efficientemente con buffering

Legge tipi di dato primitivi Java

Legge da un file

Scrive dati efficientemente con buffering

Scrive tipi di dato primitivi Java

Scrive su un file

IO DI FILE BINARI

```
public static void main(String args[]) throws IOException { 1
    FileInputStream fis = null; 2
    FileOutputStream fos = null; 3
    try { 4
        fis = new FileInputStream(args[0]); 5
        fos = new FileOutputStream(args[1]); 6
        int b; 7
        while ((b = fis.read()) != -1) { // lancia IOException 8
            fos.write(b); // lancia IOException 9
        } 10
    } catch (FileNotFoundException fnfe) { 11
        System.out.println("File not found..."); 12
    } finally { 13
        if (fis != null) fis.close(); // lancia IOException 14
        if (fos != null) fos.close(); // lancia IOException 15
    } 16
} 17
```

STREAM DI CARATTERI

`Reader` e `Writer` definiscono canali di input e output basati su caratteri

- ▶ Il costruttore apre lo stream
- ▶ Il metodo `read()` ottiene uno o più caratteri dallo stream // `Reader`
- ▶ Il metodo `write()` inserisce uno o più caratteri nello stream // `Writer`
- ▶ Il metodo `close()` chiude lo stream

Le implementazioni concrete sono specifiche del canale

- ▶ Tastiera standard, schermo, file, . . .

STREAM DI CARATTERI (CONT.)

Adatti per gestire dati testuali (tipicamente file o stringhe)

Implementazioni concrete

BufferedReader

InputStreamReader

StringReader

FileReader

BufferedWriter

OutputStreamWriter

StringWriter

FileWriter

Legge testo efficientemente con buffering

Wrapper per leggere da un canale basato su byte

Wrapper per leggere da una stringa

Legge da un file di testo

Scrive testo efficientemente con buffering

Wrapper per scrivere su un canale basato su byte

Wrapper per scrivere su una stringa

Scrive su un file di testo

IO DI FILE DI TESTO

```
public static void main(String args[]) throws IOException { 1
    BufferedReader br = null; 2
    BufferedWriter bw = null; 3
    try { 4
        br = new BufferedReader(new FileReader("i.txt")); 5
        bw = new BufferedWriter(new FileWriter("o.txt")); 6
        String line; 7
        while ((line = br.readLine()) != null) { // lancia IOException 8
            bw.write(line); // lancia IOException 9
        } 10
    } catch (FileNotFoundException fnfe) { 11
        System.out.println("File not found..."); 12
    } finally { 13
        if (br != null) br.close(); // lancia IOException 14
        if (bw != null) bw.close(); // lancia IOException 15
    } 16
} 17
```

IO DI FILE DI TESTO (CONT.)

A partire da Java 7, possiamo usare il try-with-resources

- Gli stream vengono chiusi automaticamente

```
public static void main(String args[]) { 1
    try (BufferedReader br = new BufferedReader(new FileReader("i.txt")); 2
        BufferedWriter bw = new BufferedWriter(new FileWriter("o.txt"))) { 3
        String line; 4
        while ((line = br.readLine()) != null) { 5 // lancia IOException
            bw.write(line); 6 // lancia IOException
        } 7
    } catch (FileNotFoundException fnfe) { 8
        System.out.println("File not found..."); 9
    } catch (IOException ioe) { 10
        System.out.println("Error while reading/writing the file..."); 11
    } 12
} 13
```

FILE E URL



FILE

Il package `java.io` fornisce la classe `File`

- ▶ Modella percorsi (pathname), come file e directory
- ▶ Percorsi assoluti o relativi

Fornisce metodi per gestire file e directory

- | | |
|----------------------------|----------------------------------|
| ▶ <code>create()</code> | ▶ <code>mkdir()</code> |
| ▶ <code>delete()</code> | ▶ <code>getAbsolutePath()</code> |
| ▶ <code>exists()</code> | ▶ <code>isFile()</code> |
| ▶ <code>renameTo()</code> | ▶ <code>isDirectory()</code> |
| ▶ <code>getParent()</code> | ▶ <code>listFiles()</code> |

ELENCO DEI FILE

Elenca i file contenuti nella directory corrente

```
File cwd = new File(".");  
for (File f : cwd.listFiles() // dovremmo verificare se è una directory  
    System.out.println(f.getName() + " " + f.length());  
}
```

1
2
3
4

Percorso assoluto e relativo

```
File source = new File("Main.java");  
// percorso relativo  
System.out.println(source.getName()); // 'Main.java'  
// percorso assoluto  
System.out.println(source.getAbsolutePath()); // '/path/to/Main.java'
```

1
2
3
4
5

URL

Il package `java.net` fornisce la classe `URL`

- ▶ Modella un Uniform Resource Locator (URL)
- ▶ Sostanzialmente, un puntatore a una risorsa sul web

Componenti di un URL

protocollo dominio porta percorso parametri

`https://www.google.com:443/search?q=str&cr=countryIT`

SCARICARE UN FILE

Il costruttore di URL è ora deprecato, usare invece `URI.toURL()`

► `URL www = new URI("http://info.cern.ch/index.html").toURL();`

Scaricare e stampare a schermo una pagina HTML

```
try {
    URL www = new URI("http://info.cern.ch/index.html").toURL();
    BufferedInputStream in = new BufferedInputStream(www.openStream());
    byte[] dataBuffer = new byte[1024];
    while (in.read(dataBuffer, 0, 1024) != -1) {
        System.out.println(new String(dataBuffer));
    }
} catch (IOException | URISyntaxException e) throw new RuntimeException(e);
```


COSTRUIRE URL

Costruttore alternativo della classe URI

```
String protocol = "http";           1
String host = "www.google.com";      2
String path = "/search";             3
String params = "q=str\&cr=countryIT"; 4
                                     5
URL url = new URI(protocol, null, host, -1, path, params, null).toURL(); 6
// 'http://www.google.com/search?q=str\&cr=countryIT' 7
System.out.println(url.toString()); 8
```

DATI STRUTTURATI



FILE CSV

Un file CSV (comma-separated values) è un file di testo che rappresenta dati tabulari

- ▶ Come il contenuto di un foglio di calcolo o di un database
- ▶ Formato standard, facile da parsare

Ogni file contiene una serie di righe che rappresentano le righe di una tabella

- ▶ Le colonne della tabella sono separate da virgole
- ▶ Nessun tipo di dato, tutti i valori nei file sono stringhe
- ▶ L'escaping è necessario quando i valori contengono virgole o virgolette

CSV IN JAVA

Nessun supporto nativo, è necessaria una libreria di terze parti

► Ad esempio, OpenCSV

```
<dependency>                                <!-- da aggiungere al file di configurazione Maven pom.xml -->
  <groupId>com.opencsv</groupId>
  <artifactId>opencsv</artifactId>
  <version>5.12.0</version>
</dependency>
```

La libreria fornisce le classi `CSVReader` e `CSVWriter` per leggere e scrivere file CSV

OPENCSV

Leggere un file CSV

```
// Da un oggetto File, con il reader predefinito
File csvFile = new File("i.csv");
CSVReader csvReader = new CSVReader(new FileReader(csvFile));

// Da un nome di file, con un reader personalizzato
CSVReader csvReader = new CSVReaderBuilder(new FileReader("i.csv"))
    .withSkipLines(1)
    .withSeparator(',')
    .build();

// Legge il CSV riga per riga
List<String[]> table = new ArrayList<>();
for (String[] row; (row = csvReader.readNext()) != null; ) // lancia CsvException
    table.add(row);

// Legge l'intero CSV in una volta sola
List<String[]> table = csvReader.readAll(); // lancia CsvException
```

OPENCSV (CONT.)

Scrivere un file CSV

```
// Su un oggetto File, con il writer predefinito
File csvFile = new File("o.csv");
CSVWriter csvWriter = new CSVWriter(new FileWriter(csvFile));

// Su un nome di file, con un writer personalizzato
CSVWriter csvWriter = new CSVWriter(new FileWriter("o.csv"), ';');

// Scrive il CSV riga per riga
for (String[] row : table)
    csvWriter.writeNext(row);
csvWriter.flush(); // non necessario se chiudiamo il file

// Scrive l'intero CSV in una volta sola
csvWriter.writeAll(table);
csvWriter.flush(); // non necessario se chiudiamo il file
```

FILE JSON

Un file JSON (JavaScript Object Notation) è un file di testo che fornisce una rappresentazione leggibile dall'uomo di dati strutturati

- ▶ Formato standard, facile da parsare
- ▶ Universalmente usato sul web

Ogni file contiene

// ricorsivamente

- ▶ Oggetti: coppie chiave-valore delimitate da parentesi graffe { }
- ▶ Array: liste ordinate di valori delimitate da parentesi quadre []

I valori possono essere stringhe, numeri, booleani, oggetti o array

JSON IN JAVA

Nessun supporto nativo, è necessaria una libreria di terze parti

- ▶ Ad esempio, Gson di Google

```
<dependency>                                <!-- da aggiungere al file di configurazione Maven pom.xml -->
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.13.2</version>
</dependency>
```

La libreria fornisce la classe `Gson` con i metodi `fromJson` e `toJson` per parsare e costruire contenuti JSON

- ▶ Il contenuto JSON può essere letto da o scritto su file
- ▶ Il binding con classi definite dall'utente semplifica il processo

GSON

Costruire contenuti JSON da un'istanza di una classe

```
Gson gson = new Gson();
Student sam = new Student("Sam", 123);
String json = gson.toJson(sam);
System.out.println(json); // stampa '{"matricola":123,"name":"Sam"}'
```

Parsare contenuti JSON in un'istanza di una classe

```
Gson gson = new Gson();
String json = "{\"matricola\":123,\"name\":\"Sam\"}";
Student sam = gson.fromJson(json, Student.class); // lancia JsonSyntaxException
System.out.println(sam.toString()); // stampa 'Sam 123'
```

GSON (CONT.)

Scrivere su file JSON

```
try (FileWriter writer = new FileWriter("sam.json")) {  
    Gson gson = GsonBuilder().setPrettyPrinting().create();  
    Student sam = new Student("Sam", 123);  
    gson.toJson(sam, writer); // lancia JsonIOException  
} catch (IOException ioe) {  
    throw new RuntimeException(ioe);  
}
```

Leggere da file JSON

```
try (FileReader reader = new FileReader("sam.json")) {  
    Gson gson = new Gson();  
    Student sam = gson.fromJson(reader, Student.class); // lancia JsonSyntaxException, JsonIOException  
} catch (IOException ioe) {  
    throw new RuntimeException(ioe);  
}
```



Q + A

